BCSE203E - Web Programming

Digital Assignment - I

Winter Semester 2025-2026

Topic: Develop Your Portfolio

Student Details

Name: Yash Pradhan

Registration Number: 24BCE0702

Course Code / Title: BCSE203E Web Programming

Faculty Name: Prof. Jayakumar S

Assignment Submission Data

Below are the links to the source code repositories and the live deployment of the SereniMind project.

1. Main GitHub Repository (Official Classroom Repo)

- **Link:** https://github.com/JayakumarClassroom/daily-task-yoswinner
- **Contents:** Contains the `index.html`, `style.css`, and `script.js` files for Portfolio.

2. Coursework and Project Repository

This is my comprehensive personal repository containing all lab tasks, coursework, and the final iterations of the portfolio project code.

- **Link:** https://github.com/yoswinner/Web-Programming-FS-25-26

3. GitHub Pages (Live Project Deployment)

The fully functional, live version, hosted via GitHub Pages for easy access and testing without needing a local server environment.

- **Link:** https://yoswinner.github.io/WebDev-Portfolio/

Documented Steps: Attached below.

Learning Reflections: Attached below.

Importance of Personal Webpage: Attached below.

Screenshots: Attached below.

Table of Contents

1. Step-by-Step Documentation
2. Reflection on Learning
3. The Importance of a Personal Webpage
4. Screenshots of the Webpage
5. Step-by-Step Documentation
   1.1 Pre-requisites & Tools Used
   1.2 Ideation and Design System Setup
   1.3 Semantic HTML Structuring
   1.4 Advanced CSS and Responsive Layout Design
   1.5 JavaScript Logic and Interactive Elements
   1.6 Challenges Faced and Solutions
   1.7 Version Control and GitHub Deployment
6. Reflection on Learning
7. The Importance of a Personal Webpage in the Digital Age
8. Screenshots of the Webpage

1. Step-by-Step Documentation

The objective of this assignment was to conceptualize, develop, and deploy a personal portfolio website from scratch. This documentation outlines the entire software development lifecycle of the project, from initial design architecture to final deployment via GitHub Pages.

1.1 Pre-requisites & Tools Used

Before writing any code, I established my development environment. The primary tools utilized for this project included:

- **Code Editor:** Visual Studio Code (VS Code) for writing and formatting HTML, CSS, and JavaScript.
- **Local Server:** The "Live Server" extension in VS Code to test the website locally and preview changes in real-time.
- **Version Control:** Git Bash for executing command-line instructions to track changes.
- **Hosting:** GitHub Pages for deploying the static site to a live, public domain.

1.2 Ideation and Design System Setup

The first phase was designing the visual identity. I decided on a dark-themed, futuristic aesthetic to reflect my core academic interests in Cybersecurity, Data Science, and Bioinformatics.

To maintain consistency throughout the development process, I set up a robust design system in CSS using custom properties (variables) mapped to the :root pseudo-class.

1. **Color Palette:** I utilized deep blacks (--primary-bg: #000000, --secondary-bg: #0a0a0a) combined with high-contrast neon accents (--accent-green: #00ff00, --accent-cyan: #00ffcc). This ensures that important call-to-action buttons and text highlights immediately draw the user's eye.
2. **Typography:** I imported the 'Inter' font family from Google Fonts. 'Inter' is a modern sans-serif typeface specifically designed for computer screens, ensuring excellent readability across both desktop and mobile devices.

1.3 Semantic HTML Structuring

Once the design was established, I built the index.html file. Instead of relying on generic tags, I strictly adhered to semantic HTML5 outlining.

1. The document utilizes a fixed <nav> for the header, a <main> wrapper for the core content, distinct <section> tags for each thematic area, and a <footer>.
2. I structured the content logically into sections: **Hero** (introducing my identity and 9.2 CGPA), **About** (detailing core coursework like Complex Analysis and Microprocessors), **Skills** (categorized into Programming, Domains, and Soft Skills), **Projects** (highlighting NestQuest and SereniMind), **Achievements**, **Hobbies**, and **Contact**.
3. This semantic approach ensures the webpage is accessible to screen readers and is easily

parseable by search engine web crawlers.

1.4 Advanced CSS and Responsive Layout Design

The styling phase involved translating the raw HTML into a visually engaging layout.

- **Glassmorphism:** To create a modern "tech" vibe without making the interface look cluttered, I implemented a Glassmorphism effect on the project and achievement cards. This was achieved by combining backdrop-filter: blur(10px) with semi-transparent backgrounds (rgba(255, 255, 255, 0.02)) and subtle, low-opacity borders.
- **Responsive Grids:** I utilized CSS Grid heavily to ensure the layout adapts to any screen size. By writing rules like grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)), the cards automatically reflow from a multi-column desktop layout to a single-column mobile layout without requiring excessive media queries.
- **Hover States:** I added CSS transitions to interactive elements. Hovering over a project card slightly scales it up and adds a glowing box-shadow (box-shadow: 0 0 20px rgba(0, 255, 0, 0.1)), providing instant visual feedback to the user.

1.5 JavaScript Logic and Interactive Elements

I wrote custom Vanilla JavaScript to bring the static webpage to life, significantly elevating the user experience:

- **HTML5 Canvas Animations:** I engineered a dynamic, rotating particle galaxy effect rendered on <canvas> elements for the Hero and Contact sections. This involved creating a GalaxyAnimation class that uses sine and cosine mathematical functions to plot hundreds of particles in a spiral pattern. The animation loop uses requestAnimationFrame for optimal performance.
- **Intersection Observer:** Instead of using traditional scroll event listeners (which can cause severe performance lag), I implemented the IntersectionObserver API. This API monitors when specific elements (like cards or text blocks) enter the user's viewport. Once they become visible, it triggers a smooth fade-in and translate-Y CSS animation.
- **Scroll Progress Bar:** I dynamically generated a gradient progress bar via JavaScript that attaches to the top of the screen. As the user scrolls down, a calculation based on window.scrollY and the document's total height updates the width of the bar, indicating how much of the page remains.

1.6 Challenges Faced and Solutions

- **Canvas Resizing Issue:** Initially, the canvas animations would stretch and distort if the user resized their browser window. *Solution:* I implemented a window.addEventListener('resize') function that dynamically recalculates the window.innerWidth and innerHeight, clears the canvas, and redraws the particles at the correct aspect ratio.
- **Layout Shifts on Load:** Elements with scroll animations were causing layout shifts before JavaScript was fully loaded. *Solution:* I set the initial CSS state of these elements to opacity: 0 and utilized the observer to only apply visibility once the DOM was ready and the

element was scrolled into view.

1.7 Version Control and GitHub Deployment

Throughout the development cycle, I utilized Git to manage version control, ensuring all iterative changes were securely tracked. The exact pipeline executed via Git Bash was:

1. git init: Initialized the local directory as a Git repository.
2. git add .: Staged all HTML, CSS, and JS files for the initial commit.
3. git commit -m "Initial commit: Complete portfolio UI with Canvas animations": Committed the files with a descriptive message logging the milestone.
4. git branch -M main: Set the primary branch to 'main' following modern Git conventions.
5. git remote add origin https://github.com/yoswinner/Web-Programming-FS-25-26.git: Linked my local machine repository to the remote repository hosted on GitHub.
6. git push -u origin main: Pushed the compiled code securely to the cloud.

For the final deployment, I navigated to the repository settings on the GitHub web interface, accessed the "Pages" tab, and configured the source to build from the main branch. This successfully hosted the static site live on a public .github.io domain.

2. Reflection on Learning

Building this comprehensive portfolio was an incredibly hands-on experience that bridged the gap between theoretical web programming concepts and practical software engineering. Here are 10 key technical and conceptual points I learned and mastered during the development process:

1. Mastery of HTML5 Semantic Outlining: Prior to this assignment, I often relied heavily on generic containers. Through building this portfolio, I gained a profound understanding of semantic HTML5. I learned how to use structural tags like <main>, <section>, <nav>, and <article>. This taught me that HTML is not just about placing elements on a screen; it is about describing the meaning and structure of the content for accessibility tools (like screen readers) and search engines.

2. Efficiency with CSS Custom Properties: By declaring colors and font variables in the :root pseudo-class, I learned the principles of DRY (Don't Repeat Yourself) in styling. I discovered how to create a highly maintainable stylesheet. Instead of finding and replacing hex codes across 500 lines of CSS, I can now update my entire site's color theme by changing just one variable at the top of the file.

3. Implementing the Glassmorphism UI Trend: I expanded my CSS design vocabulary by learning how to execute modern UI trends. I learned how to combine backdrop-filter: blur(), subtle borders, and low-opacity backgrounds (rgba) to create frosted-glass effects. This taught me how layering and opacity can create a sense of depth and hierarchy without relying on traditional solid shadows.

4. Mathematical Rendering via HTML5 Canvas API: Writing the galaxy background animation was a major breakthrough in my JavaScript learning. I gained practical experience using the Canvas 2D context to draw and animate hundreds of rotating particles. More importantly, it reinforced my mathematical problem-solving skills, requiring the use of Trigonometry (Math.sin and Math.cos) to plot circular and spiral coordinates dynamically on the screen.

5. Performance Optimization with Intersection Observer: Initially, I attempted to trigger fade-in animations by binding a function to the window.onscroll event. I quickly learned this is highly inefficient and causes browser lag. This project introduced me to the IntersectionObserver API, teaching me how to cleanly and efficiently execute code *only* when an element crosses the threshold of the user's viewport.

6. Dynamic DOM Manipulation: While HTML provides the skeleton, I learned how powerful JavaScript is at physically altering that skeleton after the page has loaded. I learned how to use document.createElement(), apply CSS text styles dynamically via JavaScript, and append elements like the scroll-progress bar directly to the document body without ever writing them in the index.html file.

7. Fluid Responsiveness with CSS Grid: Building a site that looks good on both a 27-inch monitor and a 6-inch mobile phone was a significant learning curve. I deepened my understanding of CSS Grid, specifically the grid-template-columns: repeat(auto-fit, minmax(...)) function. This taught me how to create layouts that are inherently fluid and self-adjusting, drastically reducing the need for rigid Media Queries.

8. Professional Git Workflow and Version Control: This assignment solidified my understanding of the standard version control pipeline. I moved from simply knowing what Git is, to comfortably using the CLI to initialize repositories, check status, stage files, write descriptive commit messages, and manage remote origins. This is a critical industry skill for collaborating on larger codebases.

9. The Mechanics of Static Site Hosting: By deploying through GitHub Pages, I learned how web hosting fundamentally works for front-end applications. I learned that GitHub servers parse the static files (HTML/CSS/JS) in the repository and serve them securely over an HTTPS connection, making the project instantly accessible worldwide without the need to configure a traditional backend server.

10. Managing the Animation Loop: When building the Canvas galaxy, I learned the critical difference between setInterval and requestAnimationFrame. I learned that requestAnimationFrame tells the browser that an animation is occurring, allowing the browser to optimize the rendering process by matching the screen's native refresh rate, resulting in significantly smoother visuals and better CPU efficiency.

3. The Importance of a Personal Webpage in the Digital Age

In the contemporary digital landscape, particularly for a student pursuing a degree in Computer Science Engineering, a personal webpage is no longer an optional luxury—it is a fundamental component of professional branding and career advancement. We exist in an era where the first interaction an employer, collaborator, or peer has with you is almost entirely digital. In this context, a personal portfolio serves as the anchor of your online identity, offering benefits that traditional application methods simply cannot match.

First and foremost, a personal webpage acts as a tangible, interactive proof of skill. While a traditional paper resume or LinkedIn profile is restricted by rigid formatting, page limits, and plain text, a portfolio is a limitless canvas. When applying for roles in web development, data science, or cybersecurity, claiming proficiency in HTML, CSS, JavaScript, or UI/UX design is standard. However, designing, coding, and deploying a responsive website from scratch serves as definitive proof of those claims. The website itself is the primary project. It demonstrates to recruiters not just *what* you know, but *how* you apply it to create real-world, functional, and aesthetically pleasing products.

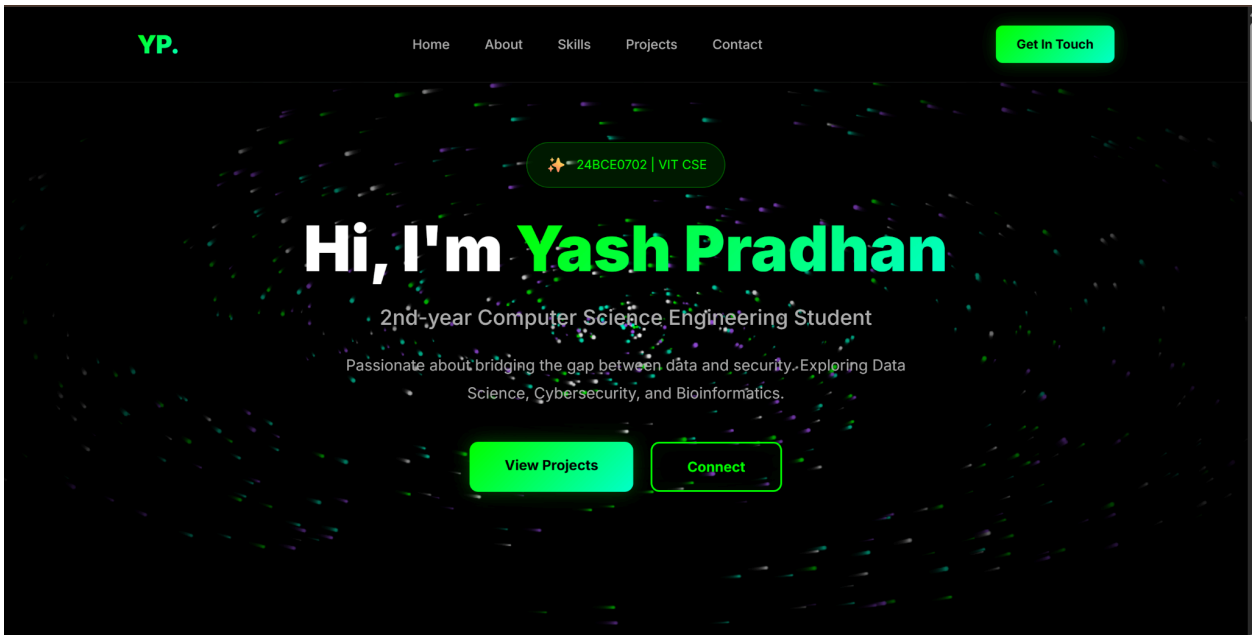
Furthermore, a personal webpage offers unparalleled freedom of expression and personal branding. Corporate resumes often strip away individuality to fit corporate standards. A personal webpage, however, allows a student to curate their exact professional narrative. For example, my portfolio highlights my 9.2 CGPA and rigorous coursework alongside my personal achievements, such as being an All India Math Topper and engaging in theatre. By controlling the design aesthetic—in my case, a dark-themed, futuristic layout—I can subtly communicate my passion for high-tech domains like Cybersecurity before the user even reads a single line of text. It humanizes the developer behind the code.

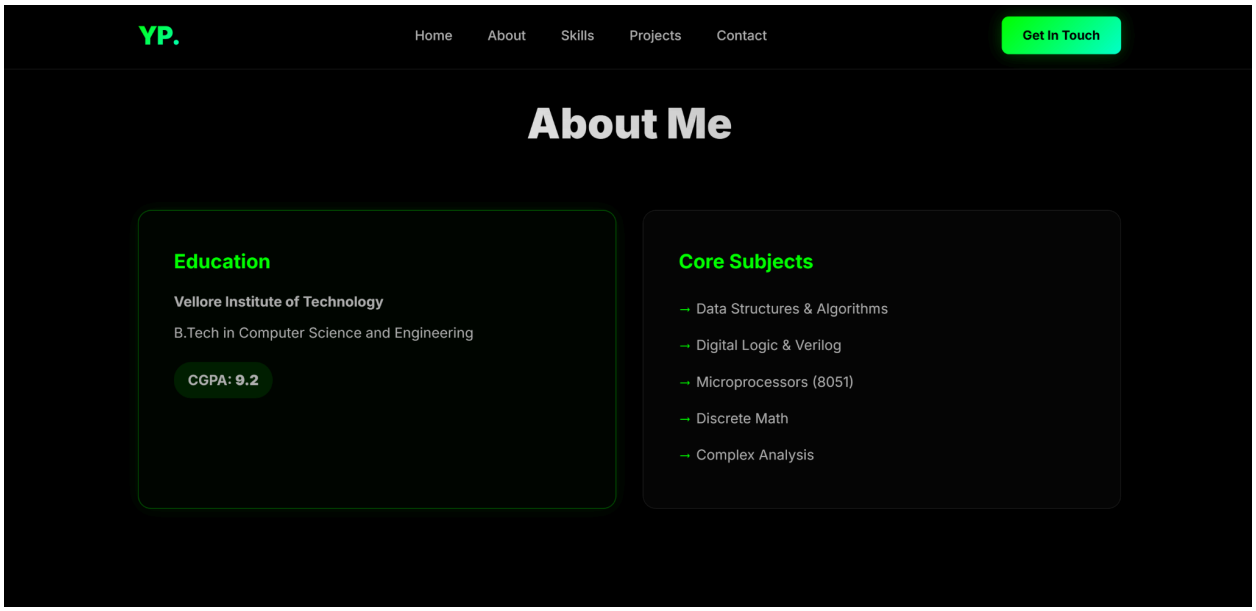
Another critical advantage is discoverability and centralization. A personal webpage acts as a centralized hub that aggregates all fragmented digital identities. It bridges the gap between my academic records, my open-source contributions on GitHub, my contact information, and detailed case studies of my projects (like NestQuest and SereniMind). By hosting this on a public domain, it becomes highly accessible. A single, clean URL can be embedded in email signatures, social media profiles, and job applications, removing all friction for someone wanting to view my body of work. Additionally, a well-structured site is indexed by search engines, meaning opportunities can find you organically.

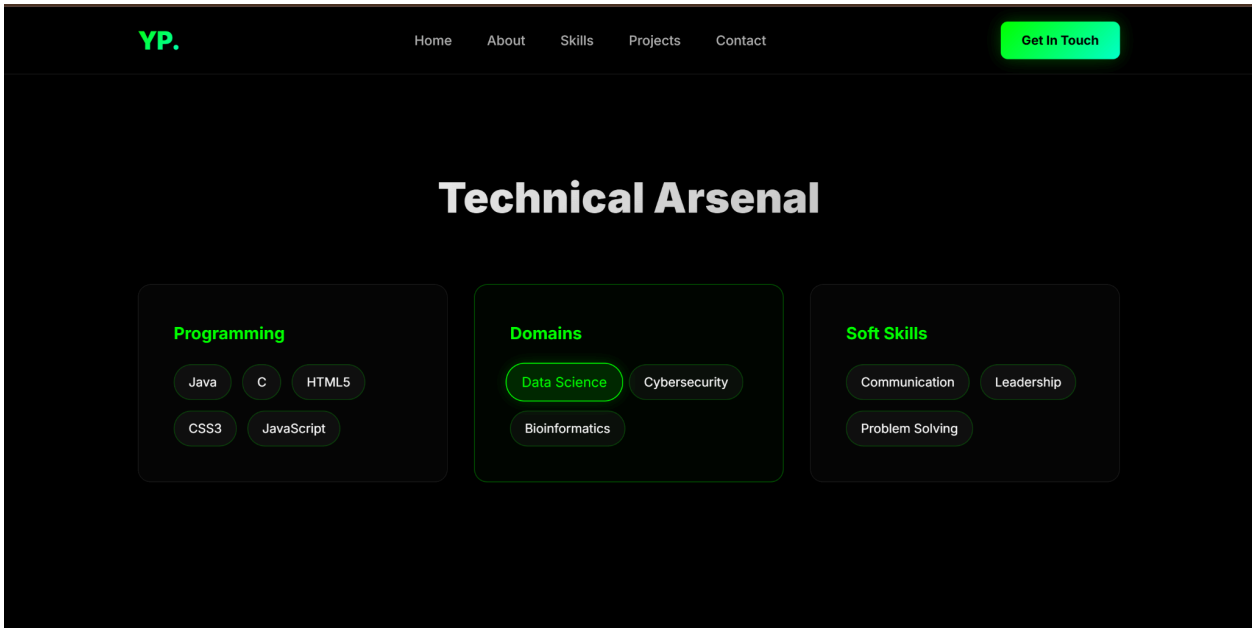
Finally, maintaining a personal webpage fosters continuous technical growth. A digital portfolio is never truly "finished." It is a living, breathing document that evolves alongside my career. Updating the site requires continuous engagement with version control systems like Git, interaction with web hosting platforms, and an ongoing understanding of modern web standards. Therefore, maintaining a personal webpage is not just an exercise in self-promotion;

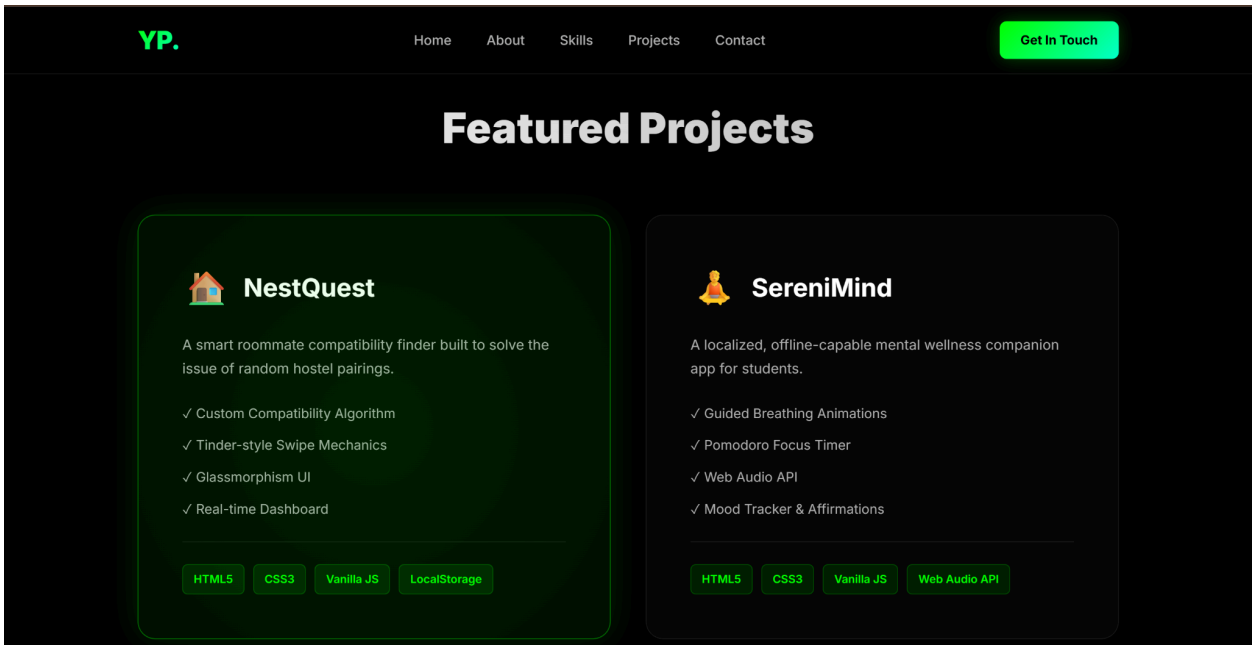
it is a proactive, career-long commitment to learning, adapting, and presenting the best possible version of my professional self to the global digital community.

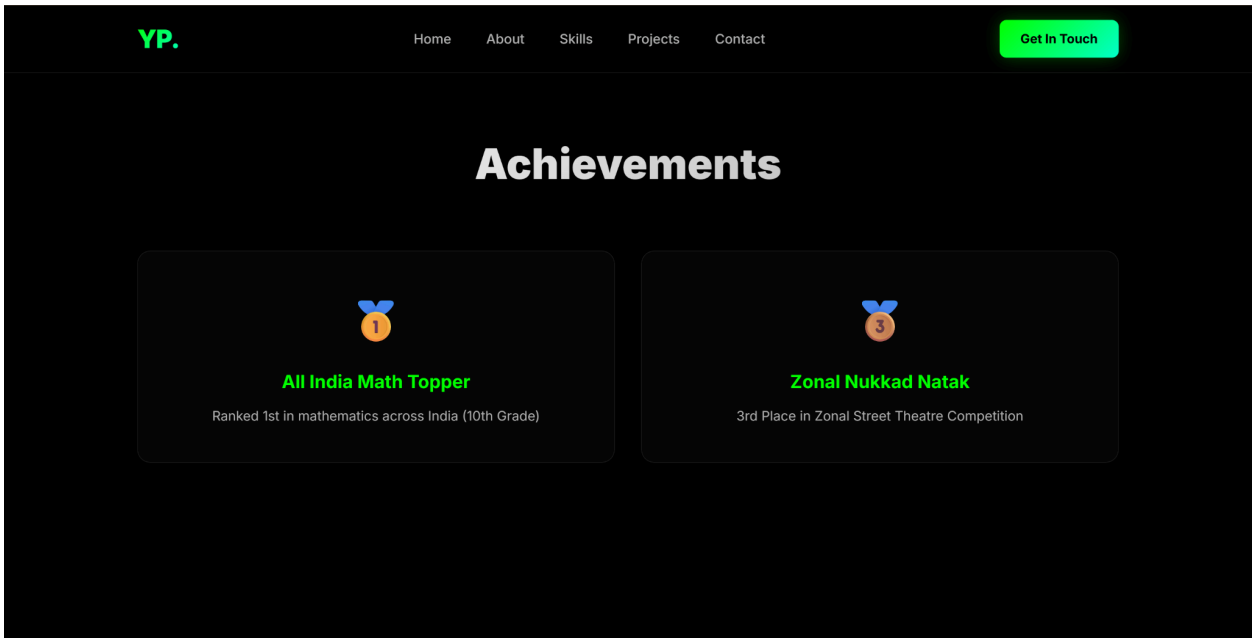
4. Screenshots

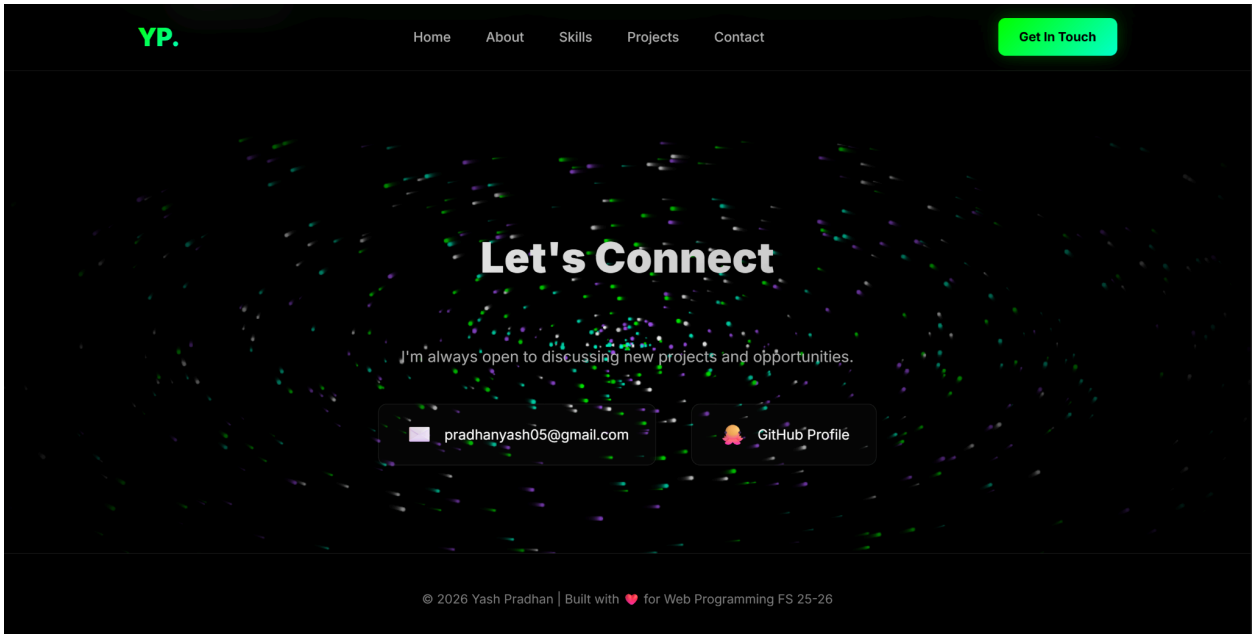





YP.
Home
About
Skills
Projects
Contact
Get In Touch
Technical Arsenal
Programming
Java
C
HTML5
CSS3
JavaScript
Domains
Data Science
Cybersecurity
Bioinformatics
Soft Skills
Communication
Leadership
Problem Solving


YP.
Home
About
Skills
Projects
Contact
Get In Touch
Featured Projects
NestQuest
A smart roommate compatibility finder built to solve the issue of random hostel pairings.
✓ Custom Compatibility Algorithm
✓ Tinder-style Swipe Mechanics
✓ Glassmorphism UI
✓ Real-time Dashboard
HTML5
CSS3
Vanilla JS
LocalStorage
SereniMind
A localized, offline-capable mental wellness companion app for students.
✓ Guided Breathing Animations
✓ Pomodoro Focus Timer
✓ Web Audio API
✓ Mood Tracker & Affirmations
HTML5
CSS3
Vanilla JS
Web Audio API


YP.
Home
About
Skills
Projects
Contact
Get In Touch
Achievements
All India Math Topper
Ranked 1st in mathematics across India (10th Grade)
Zonal Nukkad Natak
3rd Place in Zonal Street Theatre Competition


YP.
Home
About
Skills
Projects
Contact
Get In Touch
Let's Connect
I'm always open to discussing new projects and opportunities.
pradhanyash05@gmail.com
GitHub Profile
© 2026 Yash Pradhan | Built with for Web Programming FS 25-26